

数据与算法 - 第3讲

栈、递归、队列

冯伟 (fengwei@tsinghua.edu.cn)

2024年9月29日



提纲



- 栈
- 递归
- 队列

提纲



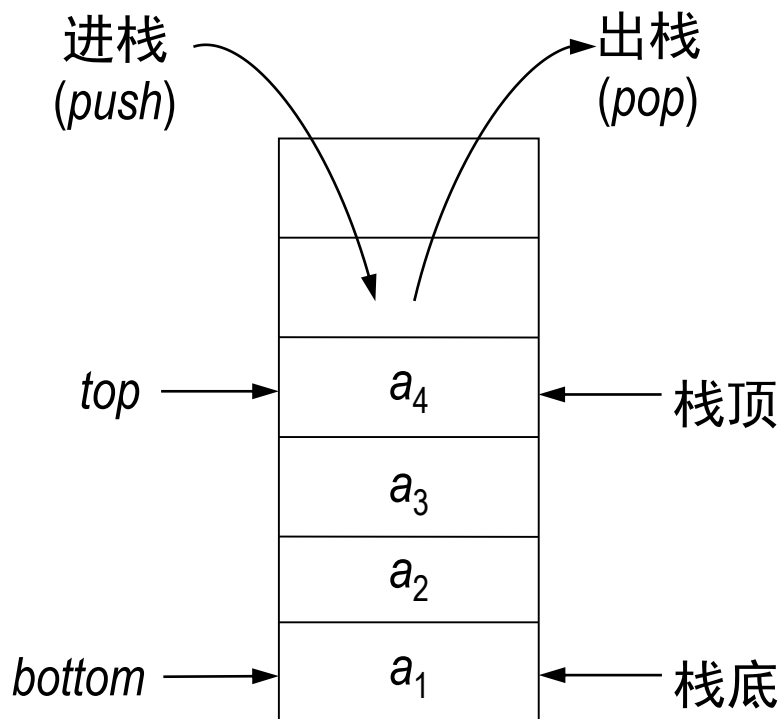
➤ 栈

➤ 递归

➤ 队列



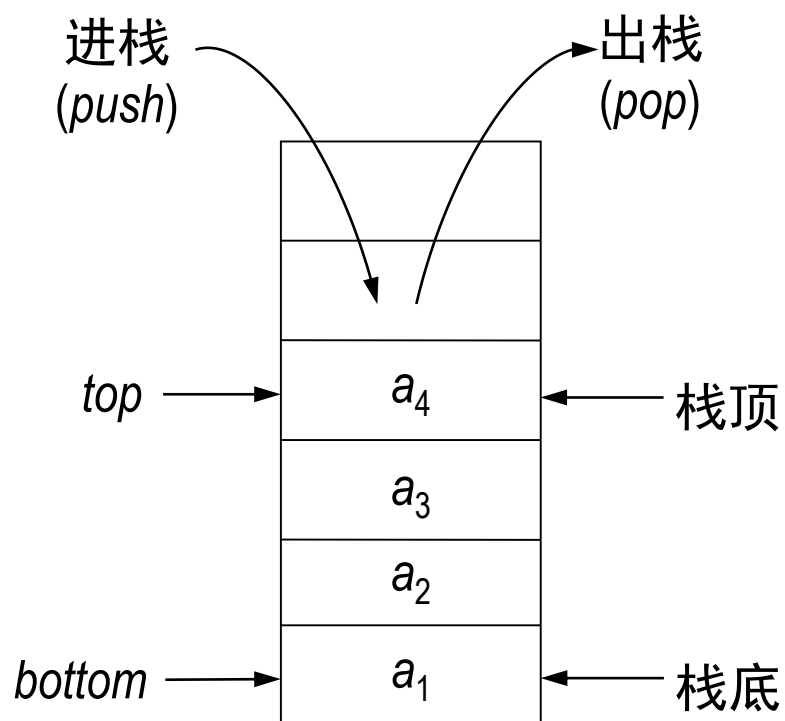
栈的概念



- 栈是一种线性结构
- 只允许在一端进行插入和删除操作
- 栈顶(Top): 允许插入和删除操作的一端
- 栈底(Bottom): 不操作的一端
- 栈中数据元素的个数就是栈的长度

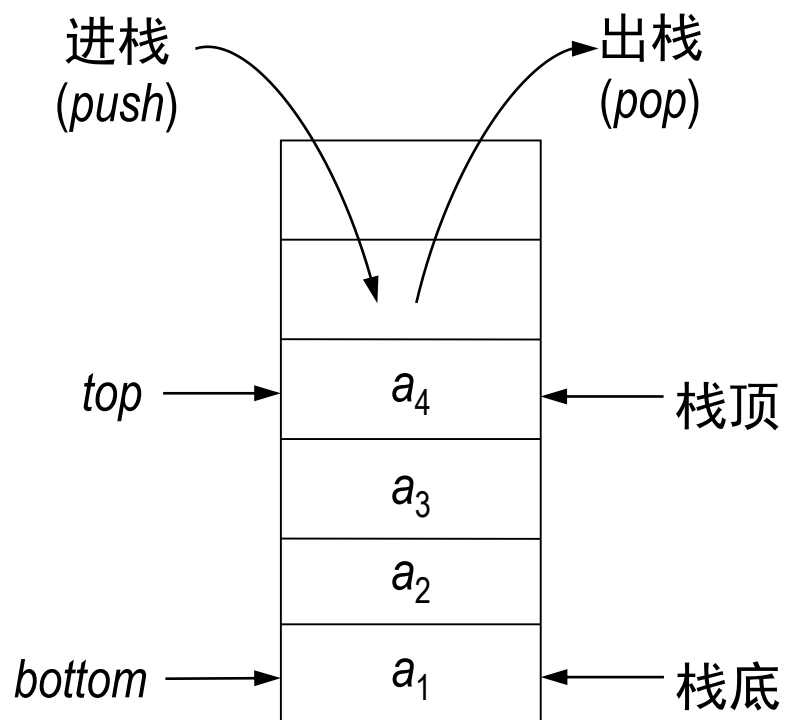


栈的概念



- 设栈为： $S = (a_1, a_2 \cdots a_n)$
- 元素的入栈顺序为：
 $a_1, a_2 \cdots a_n$ ，其中： a_1 为栈底元素， a_n 为栈顶元素
- 元素的出栈顺序为：
 $a_n, a_{n-1} \cdots a_1$
- **先进后出、后进先出**

栈的概念



- 只允许在一端进行插入和删除操作
- 先进后出、后进先出



Ctrl + Z



栈的抽象数据类型 (ADT)

ADT Stack {

数据对象: $D = \{a_i | a_i \in ElemSet, i = 1 \dots n, n > 0\}$

数据关系: $R = \{\langle a_{i-1}, a_i \rangle | a_i \in D, i = 2 \dots n\}$

约定 a_n 端为栈顶, a_1 端为栈底。

基本操作:

InitStack(&S)

操作结果: 构造一个空栈S。

DestroyStack(&S)

初始条件: 栈S已存在。

操作结果: 栈S被销毁。



栈的抽象数据类型 (ADT)

IsEmpty(S)

初始条件：栈S已存在。

操作结果：若栈S为空栈，则返回TRUE，否则FALSE。

StackLength(S)

初始条件：栈S已存在。

操作结果：返回S的元素个数，即栈的长度。

GetTop(S, &e)

初始条件：栈S已存在且非空。

操作结果：用e返回S的栈顶元素。



栈的抽象数据类型 (ADT)

Push(&S, e)

初始条件：栈S已存在。

操作结果：插入元素e为新的栈顶元素。

Pop(&S, &e)

初始条件：栈S已存在且非空。

操作结果：删除S的栈顶元素，并用e返回其值。

ClearStack(&S)

初始条件：栈S已存在。

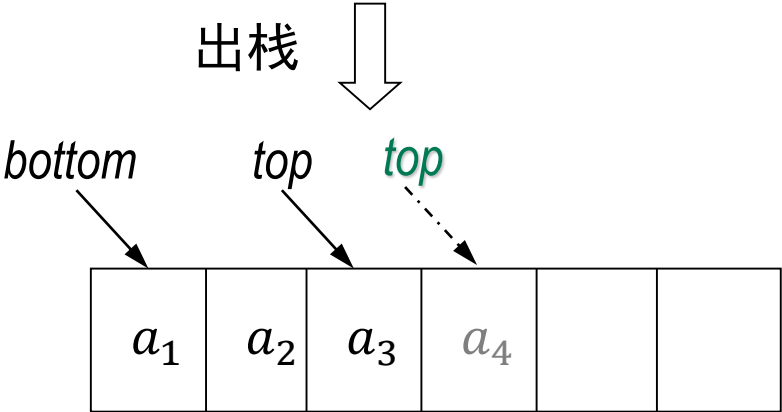
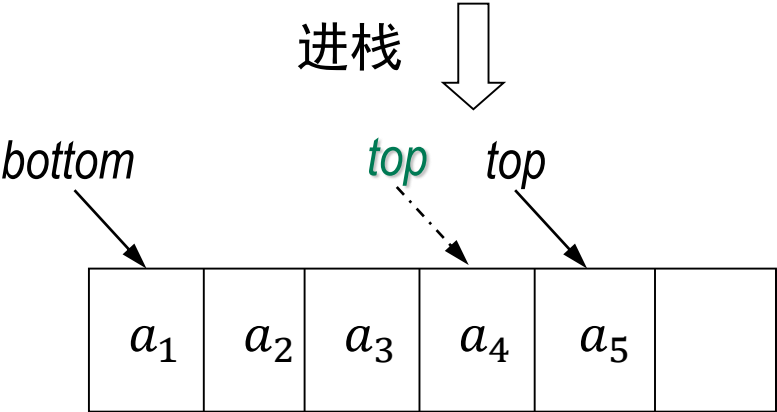
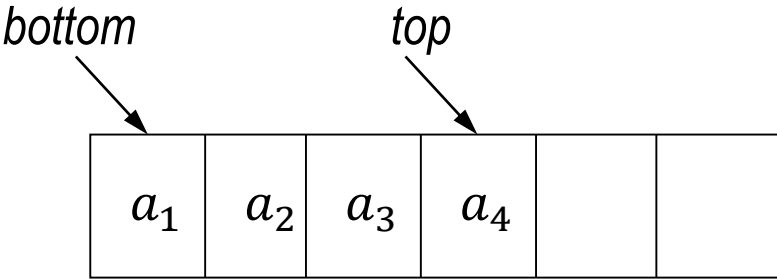
操作结果：将S清为空栈。

} ADT Stack



栈的顺序存储 - 顺序栈

基于顺序存储实现



请注意与教材p.37图2.11的不同，按应用需求设定指针位置即可



顺序栈的实现 (C++)

```
class STACK{
private:
    Item    *m_arStack;
    int     m_iDepth;
public:
    STACK(int maxLen) { m_arStack = new Item[maxLen]; m_iDepth = 0; }
    int      IsEmpty() const { return m_iDepth == 0; }
    void     ClearStack()    { m_iDepth = 0; }
    int      StackLen() const { return m_iDepth; }
    void     push(Item e)    { m_arStack[m_iDepth] = e; m_iDepth++; }
    Item     pop()           { m_iDepth--; return m_arStack[m_iDepth]; }
    Item     getTop() const  { return m_arStack[m_iDepth - 1]; }
};
```

maxLen为栈的最大规模，达到后，
执行进栈操作，出现“栈溢出”
现象，应用中跟据场景具体设计



顺序栈的效率分析

➤ 时间复杂度

- 进栈和出栈的时间复杂度都是 $O(1)$ （栈顶操作）
- 如果栈元素是简单数据类型，则构造和销毁函数也是 $O(1)$ 的

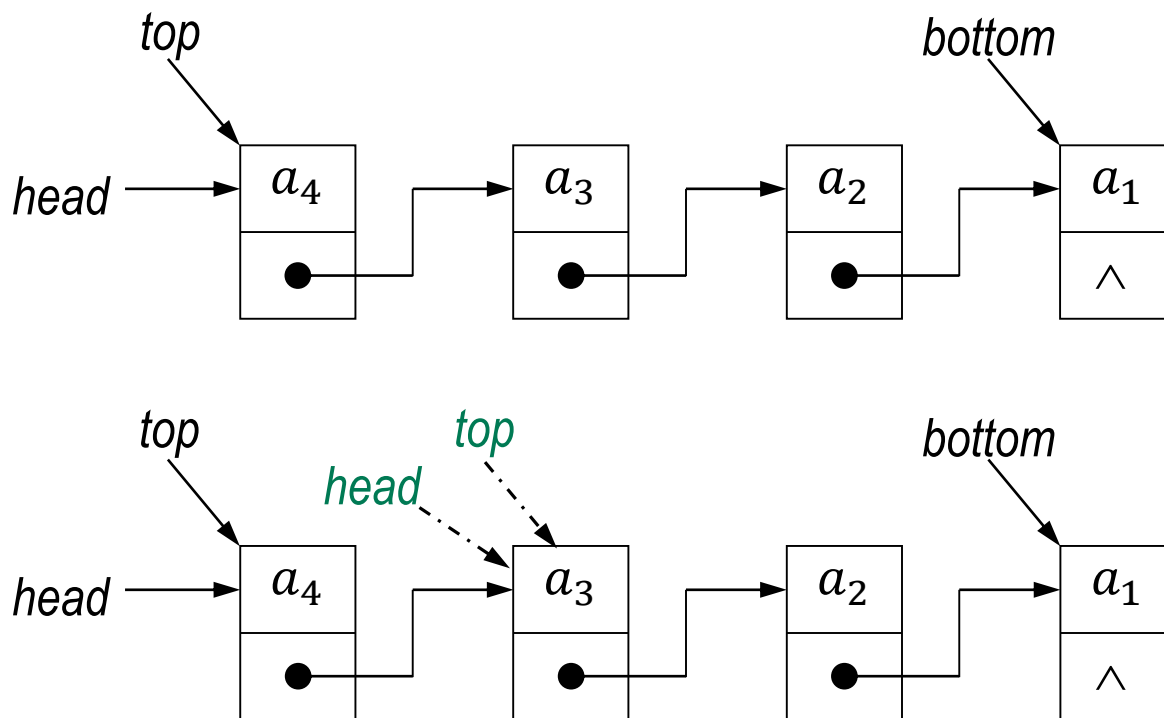
➤ 空间复杂度

- 顺序栈的长度构造时确定
- 空间利用效率较低



栈的链式存储 – 链式栈

基于链式存储实现：一般选择链表头为栈顶，链表底为栈底



出栈（入栈）操作 = 链表的头删除（头插入）操作



链式栈的实现 (C++)

```
class STACK{
private:
    link    m_head;
public:
    STACK()
    int      IsEmpty() const
    void     push(Item e)
    Item     pop()
    Item     getTop() const
    void     ClearStack( )
    StackLength( );
};

{ m_head = NULL; }
{ return m_head == NULL; }
{ m_head = new node(e,m_head); }
{ Item v = m_head->item;
  link t = m_head->next;
  delete m_head;
  m_head = t; return v; }
{ return m_head->item; }
{ while(!IsEmpty( )) pop(); }
```



链式栈的效率分析

➤ 时间复杂度

- 进栈和出栈的计算复杂度是 $O(1)$
- 建立和销毁的计算复杂度是 $O(n)$

➤ 空间复杂度

- 一般不会产生溢出
- 空间利用率高（得益于链式存储的优势）



顺序栈 vs. 链式栈

	顺序栈	链式栈
物理存储方式	地址连续	地址任意
栈的规模	初始化栈时确定	不需要事先确定
栈溢出	存在溢出可能（要处理）	一般不会溢出
操作时间复杂度	进栈/出栈: $O(1)$ 清空栈: $O(1)$	进栈/出栈: $O(1)$ 销毁和清空栈: $O(n)$
栈占用空间	初始化栈时确定栈的规模, 空间利用效率较低	栈中实际元素的数目, 空间利用效率高

栈的应用



迷宫问题 vs. 人生选择

- 栈可能是使用频率最高的数据结构
- 应用举例：**括号匹配、进制转换、表达式求值**



栈的应用 - 括号匹配

- 检查一串字符中括号是否匹配：{ ([] []) }
- 有三种括弧：{ } () []
- 操作步骤
 - 扫描字符串，逢左括弧进栈
 - 逢右括弧，将对应的左括弧出栈；
若无对应的左括弧，则为语法错误
 - 扫描字符串结束，检查栈是否为空，得到括号配对检查结果
- 可用于程序代码检查



栈的应用 - 进制转换

➤ 十进制数向其它进制转换

➤ 原理: $N = [N \text{ div } d] \times d + N \text{ mod } d$

➤ 操作步骤

- 逐次取余，依次入栈
- 依次出栈，得到结果

➤ 例 $(461)_{10} = (?)_8$

$$(461)_{10} \rightarrow (715)_8$$

8		461
8		57 5
8		7 1
8		0 7

入栈顺序: $5 \rightarrow 1 \rightarrow 7$
 $\rightarrow (715)_8$



栈的应用 – 算术表达式求值

中缀表达式	后缀表达式	前缀表达式
$A + B$	$A B +$	$+ A B$
$A \times B$	$A B \times$	$\times A B$
$A - B$	$A B -$	$- A B$
$A \times (B + C)$	$A B C + \times$	$\times A + B C$

- 不同表达式：操作数之间的相对次序不变；运算符的相对次序不同
- 后缀表达式 (把双目运算符放到两个操作数之后)
 - 表达式中不需要括号来确定优先级
 - 对计算机是最方便的求值形式



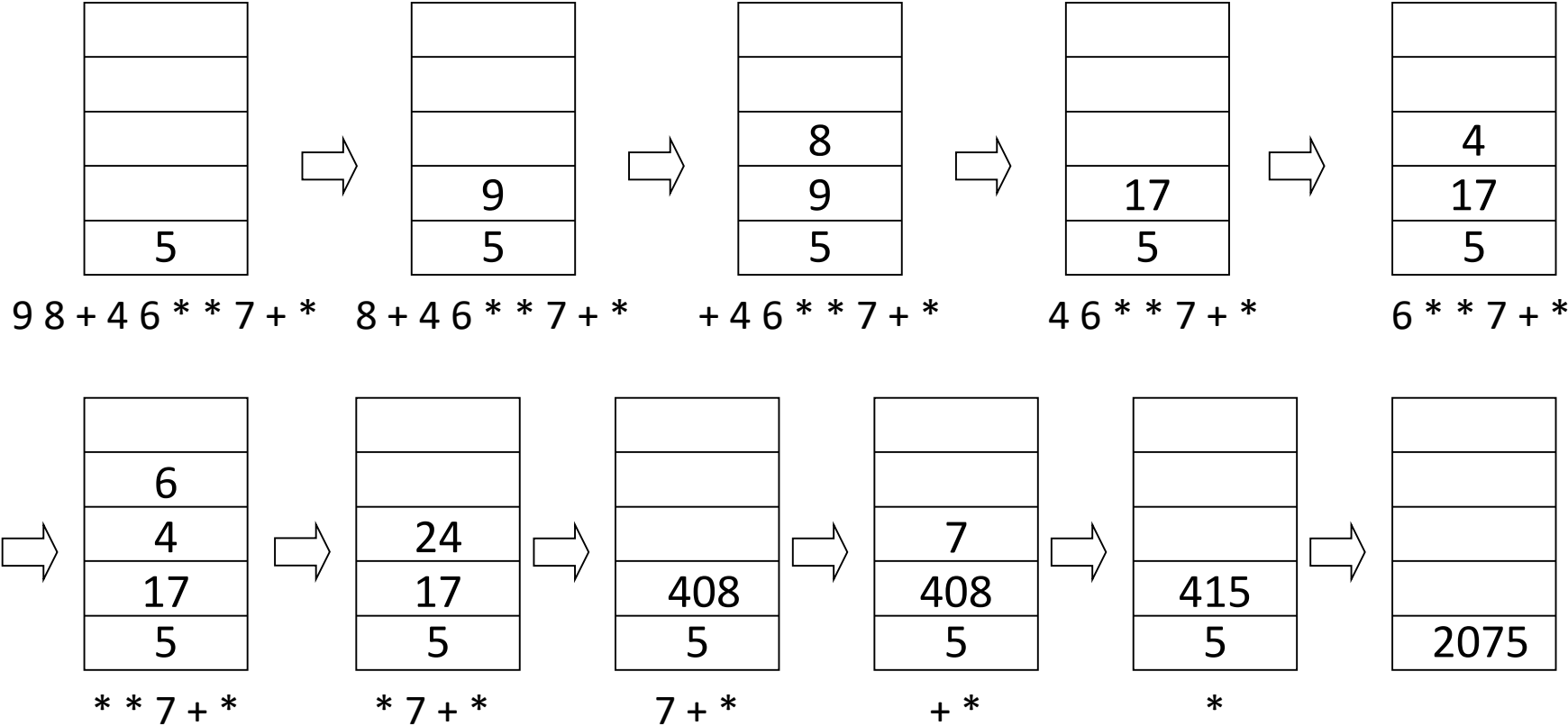
栈的应用 – 算术表达式求值

- 中缀表达式: $5 \times ((9 + 8) \times (4 \times 6)) + 7$
- 对应的后缀表达式: $5 \ 9 \ 8 \ + \ 4 \ 6 \ \times \ \times \ 7 \ + \ \times$
- 表达式计算:
 - 自左向右扫描后缀表达式
 - 遇到操作数进栈
 - 遇到操作符, 两个操作数出栈进行计算, 结果进栈
 - 栈顶元素就是后缀表达式求值的结果



栈的应用 – 算术表达式求值

求值： 5 9 8 + 4 6 × × 7 + ×





栈的应用 – 算术表达式求值

➤ 利用栈先进后出的性质，基于栈的抽象数据类型（ADT）

➤ `char* a = "5 9 8 + 4 6 × × 7 + ×"`

```
len = strlen(a);                //求表达式的长度
for(int i = 0; i < len; i++) {   //后缀表达式求值
    if(a[i] == '+') s.push(s.pop() + s.pop()); //运算符为加
    if(a[i] == '*') s.push(s.pop() * s.pop()); //运算符为乘
    if((a[i] >= '0') && (a[i] <= '9')) s.push(a[i]); //针对操作数的处理
}
```

➤ 进一步考虑：多位操作数（用栈实现，参见教材p.40的图2.14），
减法、除法（不满足交换律，除数不为0），...

注意边界、特例等 → 程序的正确性、稳健性、可读性、高效率

提纲



➤ 栈

➤ 递归

➤ 队列



递归的含义

- 若一个对象部分地包含它自己，或用它自己给自己定义，称这个对象是递归的

结构体定义链表结点

```
typedef struct node {  
    int data;  
    struct node *next;  
}NODE;
```

- 若一个过程直接地或间接地调用自己，则称这个过程是递归的



递归举例 – 阶乘计算

➤ $n! = 1 \times 2 \times 3 \cdots \times n$ ➡
$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{if } n > 0 \end{cases}$$

- 递归函数：每调用自己一次，（在某种意义上）更接近于解

```
int factorial( int n ) {  
    if(n == 0) return 1;           //基础事例  
    else return n * factorial(n - 1); //递归事例  
}
```

- 递归求解的两个条件

- 找到了规模较大的问题和规模较小的同样问题之间的联系
 - 有终止条件
- 大问题 → 小问题 → 可终止的简单问题



递归举例 – 求最大公因子

➤ 辗转相除法

- 求两个正整数 n 和 m 的最大公因子($m > n$)
- 用 n 去除 m , 将余数赋给 r
- 将 n 的值赋给 m , 将 r 的值赋给 n
- 如果 $n = 0$; 返回 m 的值作为结果, 过程结束; 否则, 返回第一步

➤ 递归实现

```
int gcd(m,n){  
    int r = m % n;  
    if(r != 0) return gcd(n,r);    //大问题到小问题  
    else      return n;           //终止条件  
}
```



递归举例 – 斐波那契数列

中世纪意大利数学家斐波那契《算盘书》中的兔子问题

- 某人买一对小兔子，养殖在完全封闭的围墙内
- 如果每对兔子每月生一对小兔子
- 小兔子出生后，二个月后具备生育能力
- 问一年后能繁衍到多少对？

月份	0	1	2	3	4	5	6	7	8	9	10	11	12
兔子对数	1	1	2	3	5	8	13	21	34	55	89	144	233



递归举例 – 斐波那契数列

➤ 斐波那契数列的递推表达式

$$F(n) = \begin{cases} 1 & n = 0, 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

➤ 斐波那契数列的通项

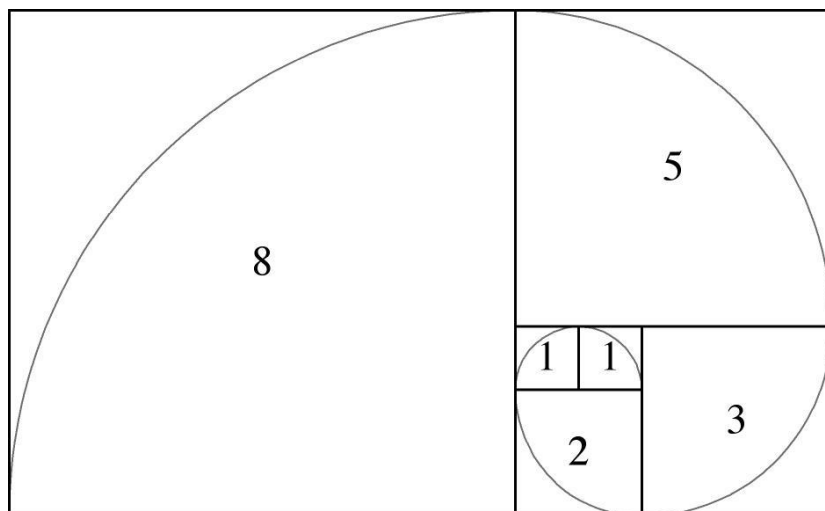
$$F(n) = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^n - \left(\frac{1 - \sqrt{5}}{2} \right)^n \right]$$

➤ 递归算法

```
long Fib( long n ) {  
    if ( n <= 1 ) return 1;           //终止条件  
    else      return Fib(n-1) + Fib(n-2); //大问题到小问题  
}
```

递归举例 – 斐波那契数列

- 在现代物理、准晶体结构、化学等领域，斐波纳契数列都有重要应用, 美国数学会设立以《斐波纳契数列季刊》为名的数学杂志
- 斐波那契日：11.23
- 斐波那契曲线

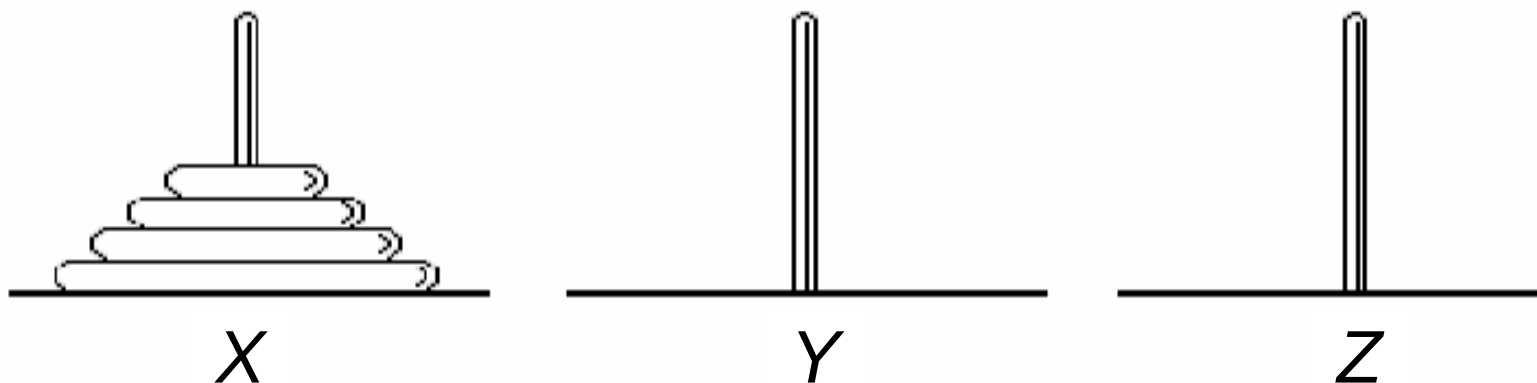




递归举例 – 汉诺塔问题

- 传说印度教主神梵天创造世界时，在印度北部佛教圣地贝拿勒斯圣庙里，安放了一块黄铜板，板上插着三根宝石针
- 在其中一根宝石针上，自下而上地放着由大到小的64个金盘。这就是所谓的梵塔(Hanoi)。梵天要求僧侣们坚持不渝地按下面的规则把64个盘子移到另一根针上
 - 一次只能移一个盘子；
 - 盘子只许在三根针上存放；
 - 永远不许大盘压小盘。
- 梵天宣称，当把他创造世界之时所安放的64个盘子全部移到另一根针上时，世界将在一声霹雳声中毁灭。而他的虔诚信徒都可以升天

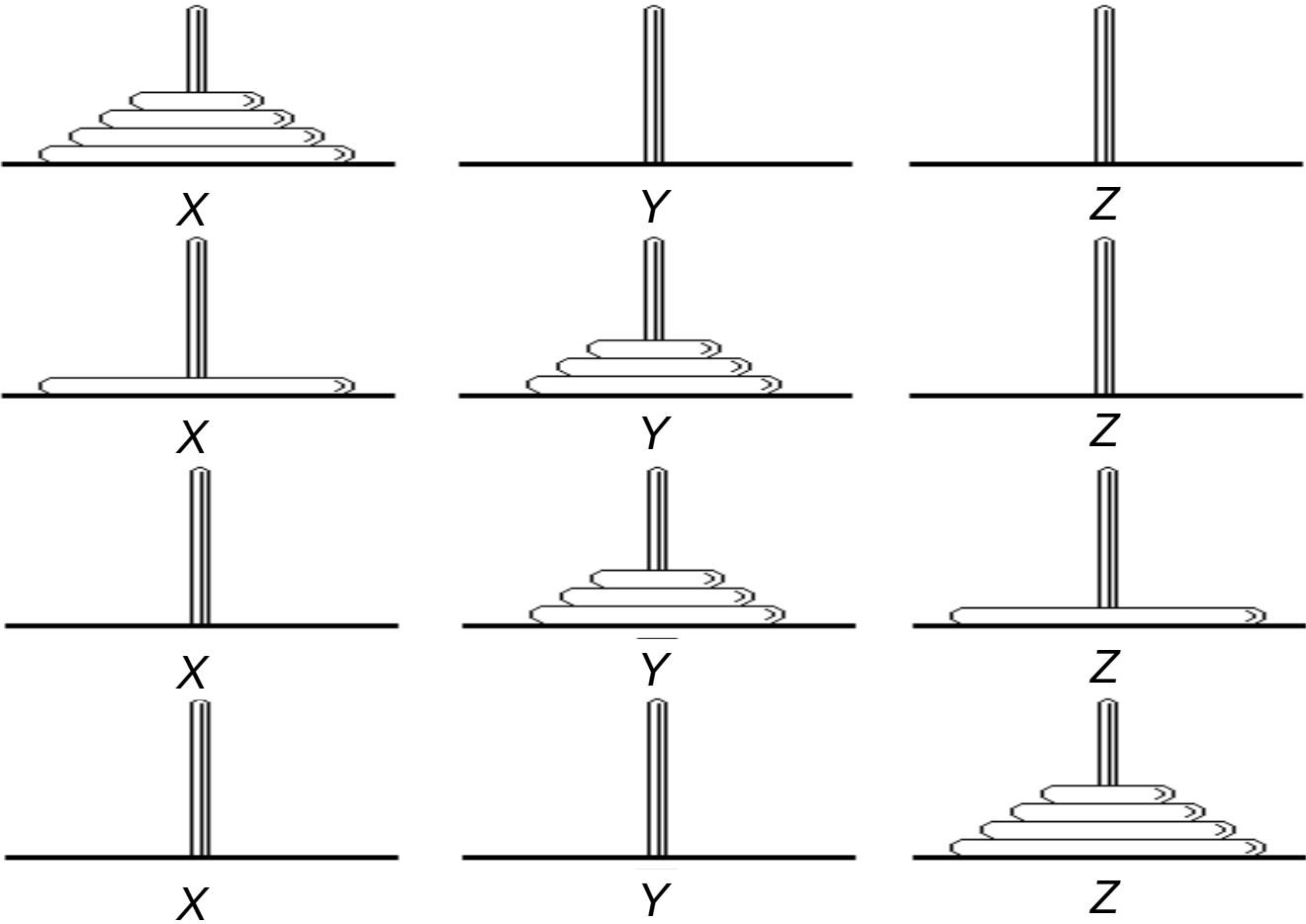
递归举例 – 汉诺塔问题



- 将塔座X上按直径由小到大的 n 个圆盘搬到塔座Z上
- Y可用作辅助塔座
- 搬运过程中，直径大的圆盘不允许在直径小的圆盘上面



递归举例 – 汉诺塔问题





递归举例 – 汉诺塔问题

- 求解 n 个圆盘的汉诺塔问题可以转化为两个 $n - 1$ 个圆盘的汉诺塔问题的求解
- 递归函数move: 把 $n + 1$ (编号0到 n) 个盘子按照规则从塔座X搬到塔座Z, Y可用作辅助塔座

```
void move( int n, int x, int z, int y) {
```

```
    if (n > 0) {
```

```
        move( n-1, x , y , z );
```

```
        printf ("Move disk %d from %d to %d ", n , x , z );
```

```
        move( n-1, y , z , x );
```

```
    }
```

```
}
```

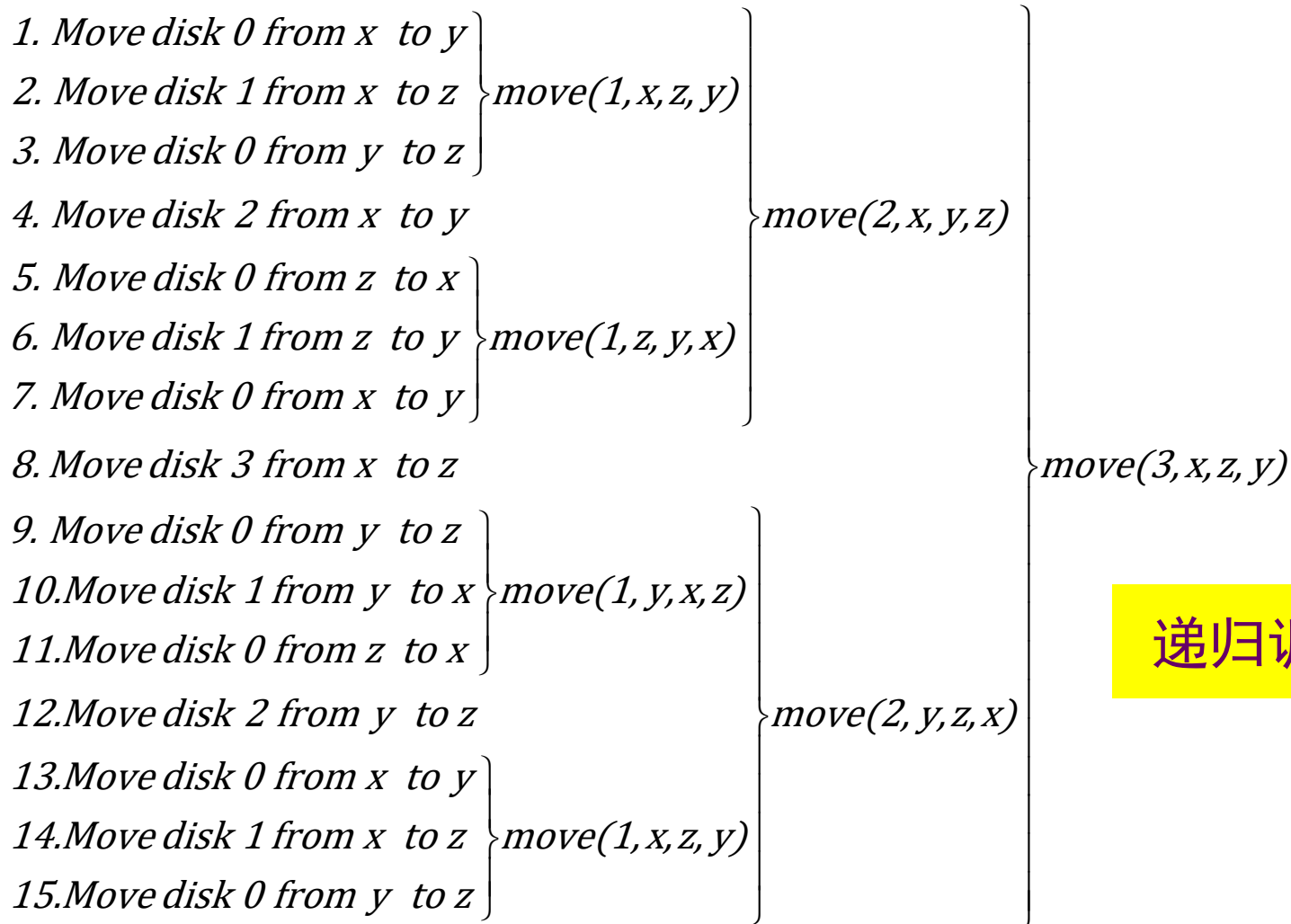
//终止条件

//大问题到小问题

//大问题到小问题



递归举例 – 汉诺塔问题



递归调用树



递归举例 – 汉诺塔问题

- 对于 n 个圆盘的汉诺塔问题，需要移动圆盘的次数为 $m(n)$ ，递推公式为

$$m(n) = \begin{cases} 0 & n = 0 \\ 2m(n-1) + 1 & n > 0 \end{cases}$$

- 令 $t(n) = m(n) + 1 = \begin{cases} 1 & n = 0 \\ 2t(n-1) & n > 0 \end{cases}$

- 递推可得： $t(n) = 2t(n-1) = 2^2t(n-2) = 2^nt(0) = 2^n$

- 进而得到： **$m(n) = 2^n - 1$**

- 金盘个数为64，则需要的移动次数为：

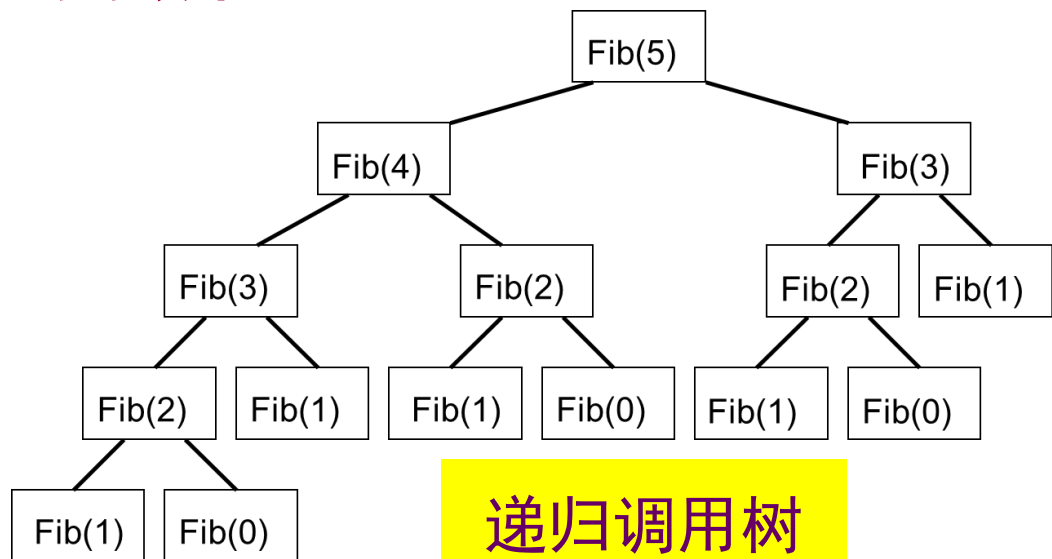
$$m(64) = 2^{64} - 1 = 1.8 \times 10^{19} \text{ (梵天常被认为是智慧之神)}$$



递归的消除

- 递归是一种有效的算法设计方法，可以大大降低求解问题的难度
- 递归算法的实现思路直接，代码简洁，易于理解
- 不足：可能导致较高的时间复杂度和空间复杂度
- 需要通过消除递归获得更高的效率

```
long Fib( long n ) {  
    if ( n <= 1 ) return 1;  
    else  
        return Fib(n-1) + Fib(n-2);  
}
```





递归的消除 – 借助栈实现非递归

- 递归：大问题 → 小问题 → 可终止的简单问题
- 依问题递减的过程，入栈保存调用现场、参数、返回地址等
- 到达终止条件后，依次出栈完成计算

$$\begin{aligned}4! \\&= 4 \times 3! \\&= 4 \times (3 \times 2!) \\&= 4 \times (3 \times (2 \times 1!)) \\&= 4 \times (3 \times (2 \times (1 \times 0!))) \\&= 4 \times (3 \times (2 \times (1 \times 1))) \\&= 4 \times (3 \times (2 \times 1)) \\&= 4 \times (3 \times 2) \\&= 4 \times 6 \\&= 24\end{aligned}$$

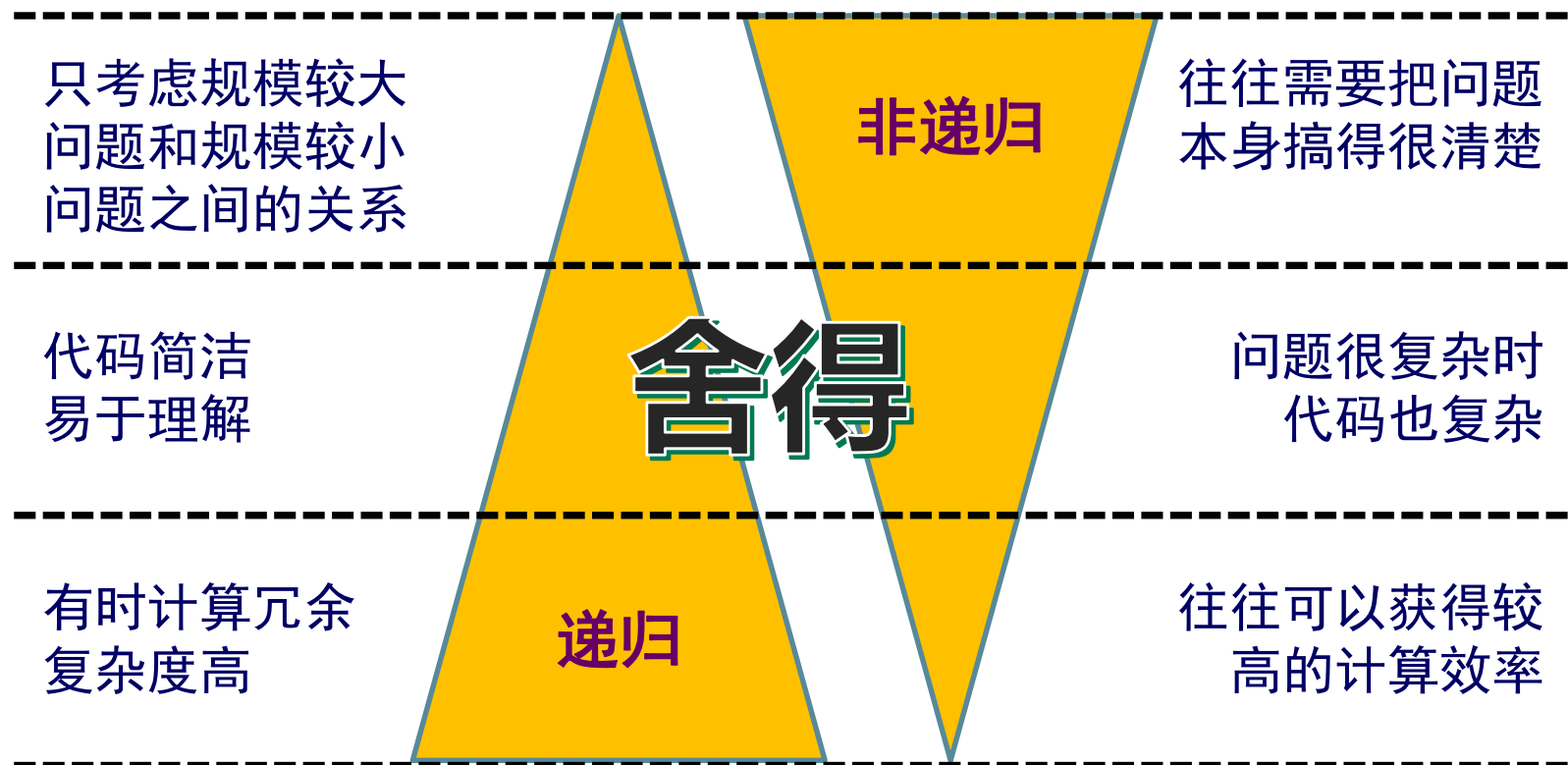
$$\begin{aligned}f(4) \\&4 \otimes f(3) \\&4 \times 3 \otimes f(2) \\&4 \times 3 \times 2 \otimes f(1) \\&4 \times 3 \times 2 \times 1 \otimes f(0) \\&4 \times 3 \times 2 \times 1 \times 1 \\&4 \times 3 \times 2 \times 1 \\&4 \times 3 \times 2 \\&4 \times 6 \\&24\end{aligned}$$

入栈

出栈



递归的消除



- 递归消除的关键在于把问题（的求解过程、细节等）搞得更清楚
- 两类比较容易消除的递归：尾递归和单向递归



递归的消除：尾递归

- 尾递归：在递归程序中，递归语句只有一个，且处在函数的最后
- 对于尾递归形式的递归算法，可以很方便的利用循环结构来替代
- 举例：阶乘求值 $n! = 1 \times 2 \times 3 \cdots \times n$

递归算法

```
int factorial( int n ) {  
    if(n == 0) return 1;  
    else return n * factorial(n - 1);  
}
```

非递归算法

```
int factorial( int n ) {  
    int a = 1;  
    while(n > 0) { a = a* n; n--;}  
    return a;  
}
```




递归的消除：单向递归

- 单向递归：递归算法中虽然有多处递归调用语句，但各递归调用语句的参数之间没有关系，并且这些递归调用语句都处在递归算法的最后
- 尾递归是单向递归的特例
- 通过设置一些变量来保存过程中的中间结果，从而用循环结构来代替递归结构

求解斐波那契数列的递归算法是单向递归

```
long Fib( long n ) {  
    if ( n <= 1 ) return n;  
    else    return Fib(n-1) + Fib(n-2);  
}
```



递归的消除：单向递归

求解斐波那契数列的非递归算法

```
long Fib_Iter ( long n ) {  
    if ( n <= 1 ) return 1;  
    long twoback = 0, oneback = 1, Current;    //定义存储变量  
    for ( int i = 2; i <= n; i++ ) {  
        Current = twoback + oneback;    //迭代公式  
        twoback = oneback;    //递推更新  
        oneback = Current;  
    }  
    return Current;  
}
```

时间复杂度为 $O(n)$



递归的消除 – 复杂度对比

- 令递归求取 n 阶斐波那契数列时，递归函数调用次数为 $C(n)$ ，则用数学归纳法可证明： $C(n) = 2 \times F(n) - 1$

□ 初有： $C(0) = 2 \times F(0) - 1 = 1$ ， $C(1) = 2 \times F(1) - 1 = 1$

□ 依归纳法，推导可得：

$$\begin{aligned} C(n) &= C(n-1) + C(n-2) + 1 = 2 \times F(n-1) + 2 \times F(n-2) - 1 \\ &= 2 \times [F(n-1) + F(n-2)] - 1 = 2 \times F(n) - 1 \end{aligned}$$

- 由于 $F(n) = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right]$ ，因此递归函数调用次数为 $O(t^n)$
- 非递归算法（递推法）的时间复杂度为 $O(n)$

提纲



➤ 栈

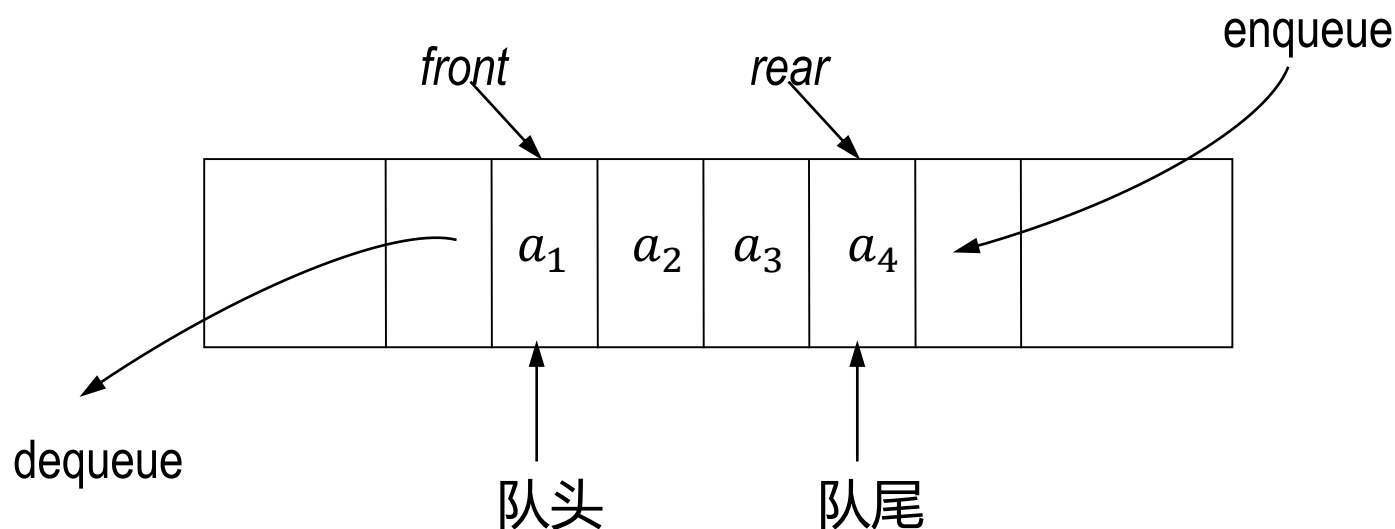
➤ 递归

➤ 队列



队列的概念

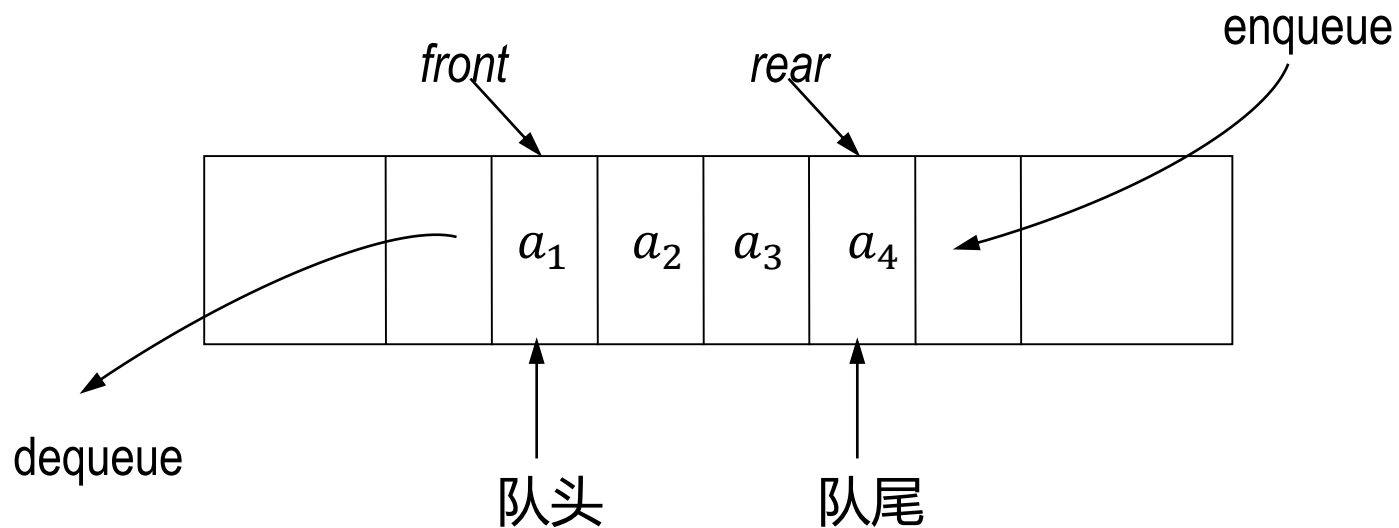
- 队列是限定在表的一端进行插入，而在另一端进行删除的线性结构
- 在队列中，允许插入的一端称为队尾(*rear*)，允许删除的一端称为队头(*front*)，插入操作称为入队(*enqueue*)，删除操作称为出队(*dequeue*)





队列的概念

- 设队列为: $Q = (a_1, a_2, \dots, a_n)$, 称 a_1 为队头元素, a_n 为队尾元素
- 队列中元素按 $a_1, a_2, \dots, a_n$ 的顺序入队(enqueue), 沿着相同的方向 $a_1, a_2, \dots, a_n$ 顺序出队(dequeue)
- **队列是先进先出的线性表**

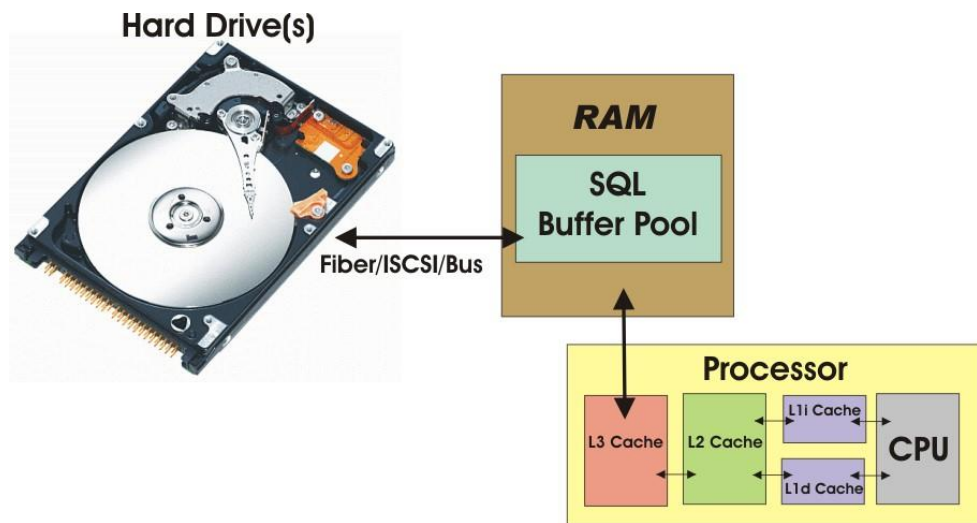


队列的概念 - 用于构建缓冲区

队列先进先出的特点可用于构建缓冲区，以处理不同设备（CPU是高速设备、键盘、打印机等是慢速设备）之间的速度差异，避免数据丢失



<http://webhtml.horhook.com/>



<https://logicalread.com/>



队列的抽象数据类型(ADT)

ADT Queue{

数据对象: $D = \{a_i | a_i \in ElemSet, i = 1, 2 \dots n, n > 0\}$

数据关系: $R = \{\langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i = 2 \dots n\}$

约定其中 a_1 端为队列头, a_n 端为队列尾。

基本操作:

InitQueue(&Q)

操作结果: 构造一个空队列Q。

DestroyQueue(&Q)

初始条件: 队列Q已存在。

操作结果: 队列Q被销毁。



队列的抽象数据类型(ADT)

IsEmpty(Q)

初始条件：队列Q已存在。

操作结果：若Q为空队列，则返回TRUE，否则返回FALSE。

QueueLength(Q)

初始条件：队列Q已存在。

操作结果：返回Q的元素个数，即队列的长度。

GetHead(Q, &e)

初始条件：队列Q存在且非空。

操作结果：用e返回Q的队头元素。

ClearQueue(&Q)

初始条件：队列Q已存在。

操作结果：将Q清为空队列。



队列的抽象数据类型(ADT)

EnQueue(&Q, e)

初始条件：队列Q已存在。

操作结果：插入元素e为Q的新的队尾元素

DeQueue(&Q, &e)

初始条件：队列Q存在且非空。

操作结果：删除Q的队头元素，并用e返回其值

QueueTraverse(Q, visit())

初始条件：队列Q存在且非空。

操作结果：从队头到队尾依次对Q的每个数据元素调用函数visit

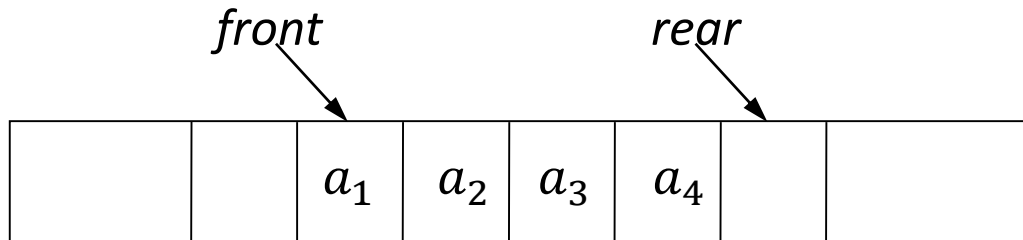
} ADT Queue



队列的顺序存储：顺序队列

队列的实现一般采用顺序存储方式

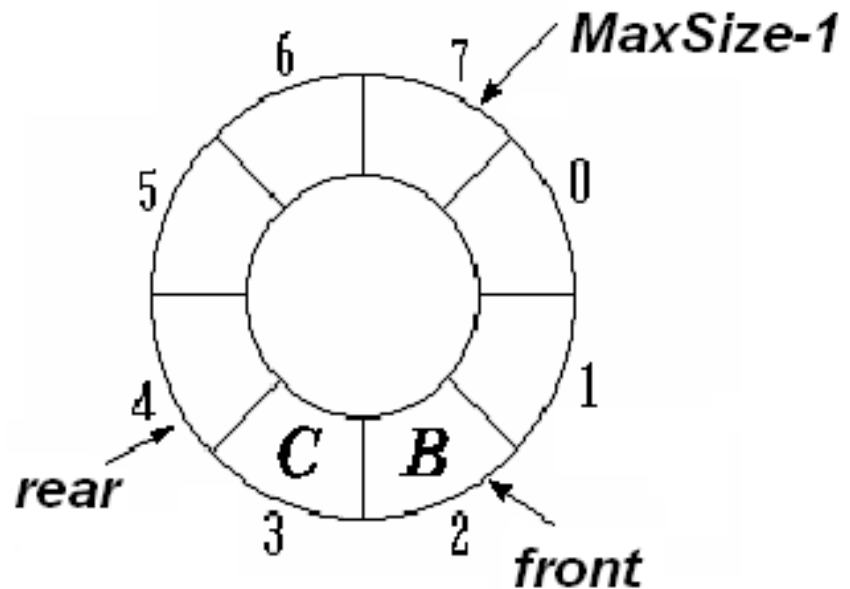
- 入队时将新元素按 $rear$ 指示位置加入，队尾指针进一： $rear = rear + 1$
- 出队时将下标为 $front$ 的元素取出，队头指针进一： $front = front + 1$
- 特殊情况：队满时再入队、队空时再出队
- 反复进行入队、出队操作，队列中的数据就会自左向右移动，左侧的存储单元不能再被使用



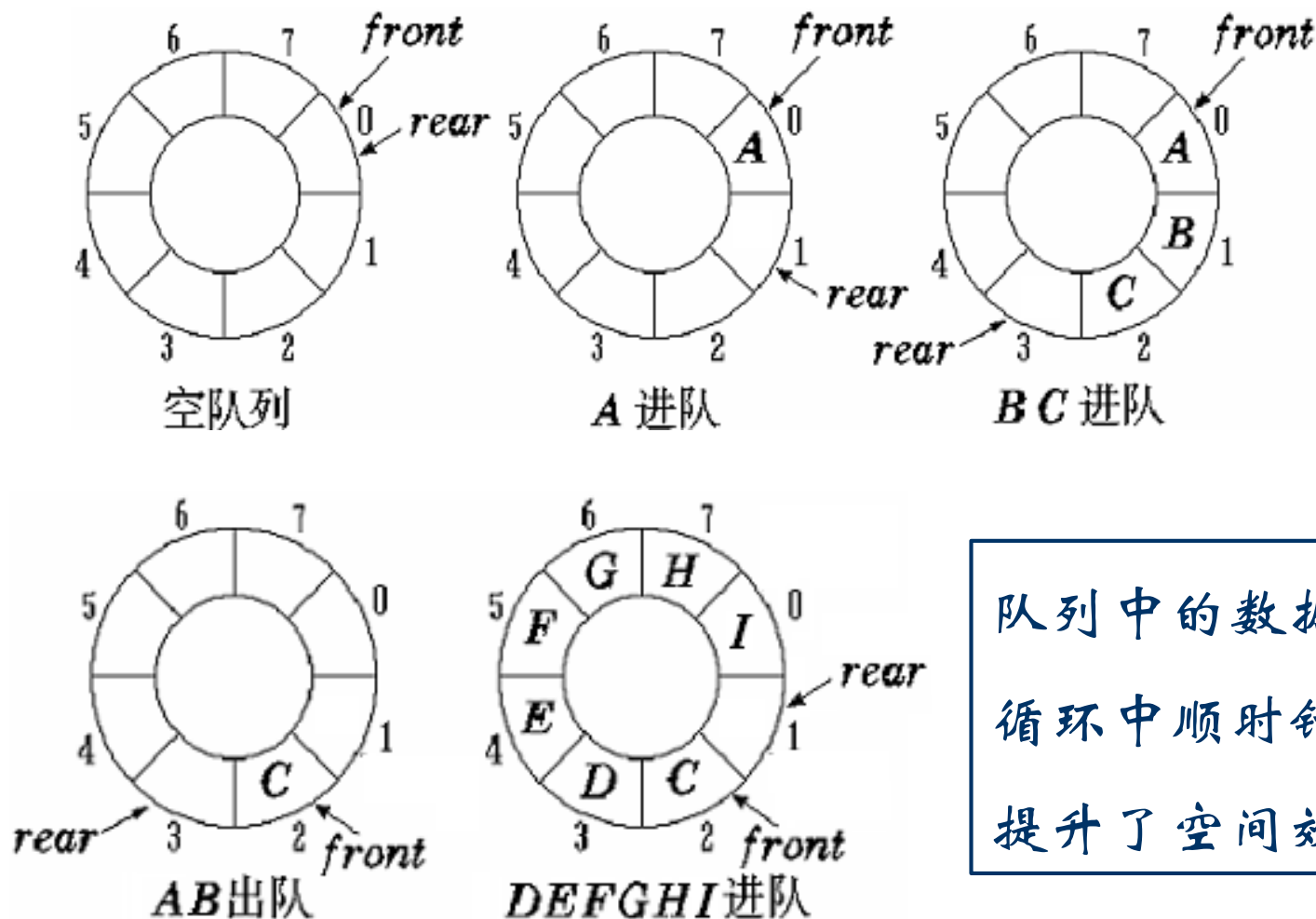


队列的顺序存储：循环队列

- 存储队列的数组被当作首尾相接的表处理；
- 假设队列的最大长度为maxSize
- 当队尾指针指向maxSize -1时，再入列可直接进到0；
- 当队头指针指向maxSize -1，再出列也直接进到0；



队列的顺序存储：循环队列



队列中的数据在
循环中顺时针移动
提升了空间效率！



队列的顺序存储：循环队列

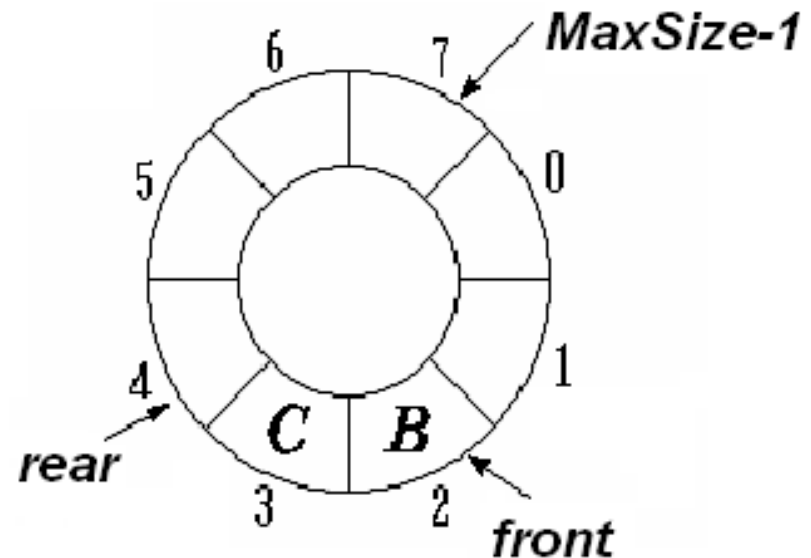
- 为了适应数据的循环移动，修改入队、出队操作：

dequeue: front $\Leftarrow$ (*front* + 1) *mod n*

enqueue: rear $\Leftarrow$ (*rear* + 1) *mod n*

- 队列中数据元素个数为：

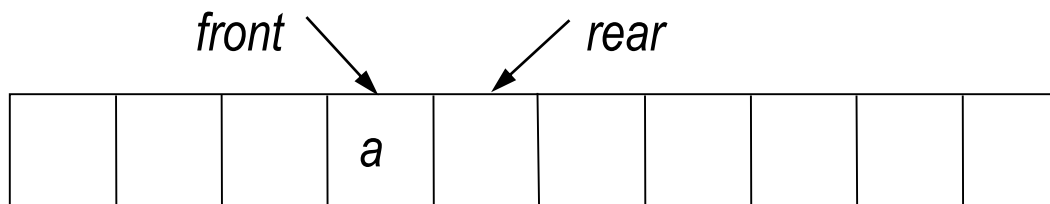
$(rear + n - front) \bmod n$



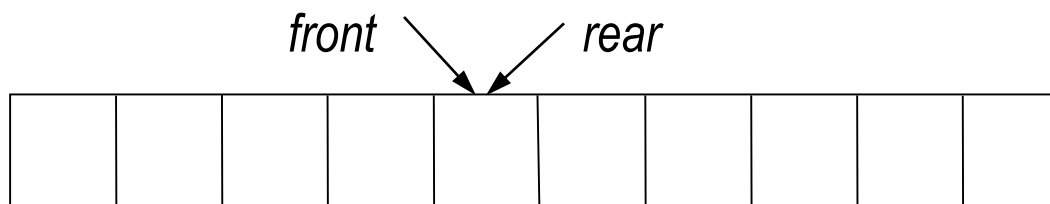


队列的顺序存储：循环队列

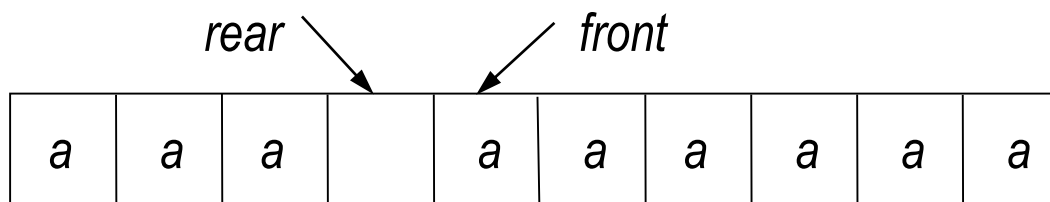
仅有一个元素



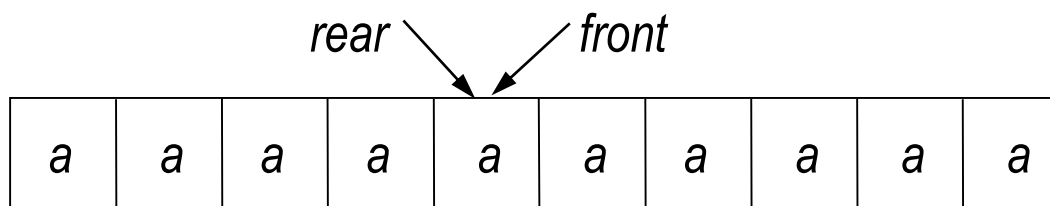
空队列



仅有一个空位



满队列



当 $front=rear$ 时，无法区分循环队列是空还是满！



队列的顺序存储：循环队列

区分循环队列是空还是满的方法

- 设置一个空位：当只有一个空位时，认为队列已满
 - 队列空： $front = rear$
 - 队列满： $front = (rear + 1) \bmod n$
- 设置一个标志
 - 队列空：由于出队操作，改变了 $front$ 指针，使得 $front = rear$ ；
 - 队列满：由于入队操作，改变了 $rear$ 指针，使得 $front = rear$ ；
- 设置并维护队列长度变量：入队+1，出队-1

引入额外的存储单元和操作



循环队列的顺序存储实现（C++）

```
class CirQueue{
private:
    int    *array, front, rear           //存储区指针以及队头队尾指针
    int    len, status;                 //队列容量和队列状态
public:
    CirQueue(int length);               //构造函数
    ~CirQueue();                        //析构函数
    void    EnQueue(int elem);           //元素进队
    void    DeQueue(int *elem);          //元素出队
    int     IsEmpty() const;             //队列判空
    int     IsFull() const;              //队列判满
    int     Length() const;              //队列长度
    void    traverse(void (*visit)(int elem)); //遍历队列所有元素
    friend void visit(int elem);         //访问操作为友元函数
};
```



循环队列的顺序存储实现 (C++)

```
CirQueue::CirQueue(int length){  
    array = new int[length];  
    len = length;  
    front = 0;    rear = 0;  
    status = EMPTY;  
}
```

//构造函数
//分配内存
//队列容量
//队头, 队尾指针
//队列为空

```
CirQueue::~~CirQueue()    {delete array;}  
int CirQueue::IsEmpty() const {return status == EMPTY;}  
int CirQueue::IsFull()    const {return status == FULL;}  
int CirQueue::Length()    const {  
    return (rear - front + len)%len;  
}
```

//析构函数
//队列判空
//队列判满
//求队列长度



循环队列的顺序存储实现 (C++)

```
void CirQueue::EnQueue(int elem){  
    if(IsFull()) cout<< "Queue Full" << endl;  
    array[rear] = elem;  
    rear = (rear + 1)%len;  
    if(rear == front) status = FULL;  
    else      status = READY;  
}  
  
void CirQueue::DeQueue(int *elem){  
    if(IsEmpty()) cout<< "Queue Empty"<< endl;  
    *elem = array[front];  
    front = (front + 1)%len;  
    if(rear == front) status = EMPTY;  
    else      status = READY;  
}
```

//元素进队操作

//队列已满

//元素赋值

//队尾指针进一

//队列状态判断

//元素出队操作

//队列为空

//元素赋值

//队头指针进一

//队列状态判断



循环队列的顺序存储实现 (C++)

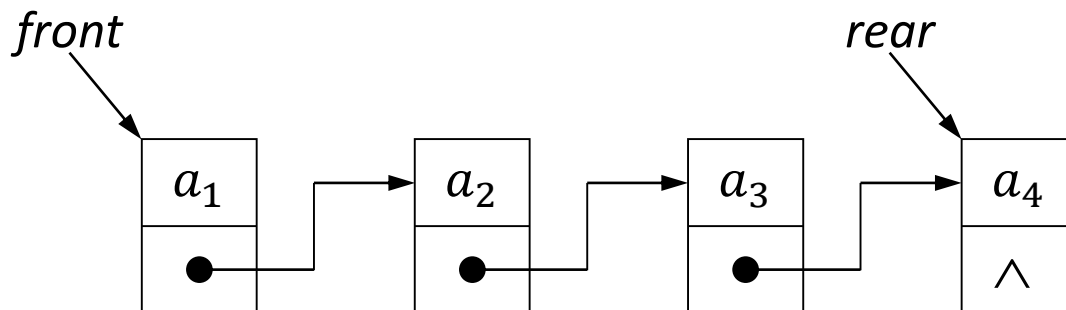
```
void CirQueue::traverse(void (*visit)(int elem)){           //遍历队列元素
    if(IsEmpty()) return;                                   //队列为空
    int p = front;                                          //从队头开始遍历
    do{
        visit(array[p]);                                    //访问操作
        p = (p+1)%len;                                     //访问位置进一
    } while(p != rear);
}

void visit(int elem) {cout << elem << "\t";}              //访问操作
```



队列的链式存储：链式队列

- 队列也可采用链式存储，构成链式队列
- 队头在链表头，队尾在链表尾：**只在头尾操作的更简单的链表**
- 入队操作：在链表尾部新加入一个结点，并把队尾指针指向这个新结点
- 出队操作：取出当前队头指针指向的链表节点，并将队头指针指向其后继结点，队空条件为 $front = NULL$





链式队列的实现 (C++)

```
class LinkQueue{  
private:  
    NODE *front, *rear;  
public:  
    LinkQueue();                //构造函数  
    ~LinkQueue();              //析构函数  
    void EnQueue(int elem);     //元素进队  
    void DeQueue(int *elem);    //元素出队  
    int  IsEmpty() const;       //队列判空  
    int  Length() const;        //队列长度  
    void traverse(void (*visit)(int elem)); //队列遍历  
    friend void visit(int elem); //访问操作  
};
```



链式队列的实现（C++）

```
LinkQueue::LinkQueue() {front = NULL; rear = NULL;}
```

//构造函数

```
LinkQueue::~LinkQueue(){
```

//析构函数

```
    NODE *p;
```

```
    while(front) { p=front;    front = front->next; delete p;}
```

```
}
```

```
Int LinkQueue::IsEmpty() const {return front == NULL;}
```

//队列判空

```
Int LinkQueue::Length() const {
```

//求队列长度

```
    if(front == NULL) return 0;
```

//队列为空

```
    int i=1;
```

```
    NODE *p = front;
```

```
    while(p!= rear) {p = p->next; i++;} return i;
```

//沿指针计数

```
}
```



链式队列的实现 (C++)

```
void LinkQueue::EnQueue(int elem) {
```

//元素进队操作

```
    NODE *p = new NODE[1];
```

//构造新结点

```
    p->data = elem;
```

//新结点赋值

```
    p->next = NULL;
```

```
    if(!front) { front = p;      rear = p; }
```

//队列为空的情况

```
    else {rear->next = p;  rear = rear->next; }
```

//队列不为空的情况

```
}
```

```
void LinkQueue::DeQueue(int *elem) {
```

//元素出队操作

```
    if(!front) cout << "queue is empty" << endl; return;
```

//队列为空

```
    NODE * p = front;
```

//取元素值

```
    *elem = p->data;
```

//删除结点

```
    front = front->next;
```

```
    delete p; p = NULL;
```

//销毁，防止内存泄漏

```
}
```




链式队列的实现 (C++)

```
void LinkQueue::traverse(void (*visit)(int elem)){
```

//队列的遍历

```
    if(front == NULL) return;
```

//队列为空

```
    NODE *p=front;
```

```
    while(p != rear) {
```

```
        visit(p->data);
```

//访问操作

```
        p = p->next;
```

//访问位置进一

```
    }
```

```
    if (p) visit(p->data);
```

```
}
```

```
void visit(int elem) {cout << elem << "\t" ;}
```

//访问操作



本讲小结

- 栈：只允许在一端进行插入和删除操作，先进后出
- 队列：一端进行插入，另一端进行删除，先进先出
- 递归：利用规模较大的问题和规模较小的同样问题之间的联系，以及终止条件，自己调用自己地解决问题
- 特殊情况的考虑：空、满、分不清、……
- 效率考虑：静态往往效率较低，动态往往比较麻烦，合适的就是最好的



一首打油诗，与同学们共勉 😊

选择

排别人的队列
修自己的栈
赢了，却出了
坚持着，竟沉淀了
队列易散
人去栈空
有的，溢出在思念里
有的，亏欠于指划中

——2021年9月作于清华园备课时

加油同学们!

